

\

Guía de Despliegue — Migración a PC Remota vía AnyDesk

Objetivo: instalar el sistema Python ([migration/](#)) en la PC del cliente, migrar los datos reales de [fcmenu.mdb](#) (Access) a PostgreSQL, y dejar el sistema operativo — de la forma más rápida, segura, simple y prolija posible, considerando que todo el trabajo se hace en remoto vía AnyDesk.

Estado del repositorio al momento de escribir esta guía: rama [main](#), último commit [e478d87](#) ("Cablear AfipWSFEv1Cliente real en las 3 ventanas"). El circuito AFIP real (TRA→firma→SOAP→CAE) ya fue validado en Homologación ([0c9134b](#), "PRIMER CAE APROBADO de verdad"). Hay cambios sin commitear en [migration/etl/export_access_to_csv.ps1](#), [load_csv_to_postgres.py](#), [models.py](#), [pdf.py](#) y [repository.py](#) (el trabajo de repetibilidad del ETL + corte de 5 años ya resuelto en VSCode) — **el primer paso de esta guía es dejar eso commiteado y etiquetado**, no llegar a la PC remota con trabajo suelto.

Arquitectura confirmada de este despliegue — tres máquinas distintas, no dos:

Máquina	Rol	Qué se instala/toca ahí
PC A (del cliente)	Corre HOY el sistema legacy VB6 con el fcmenu.mdb real	Nada. No se instala nada nuevo, no se toca el .mdb original — sólo se saca una <i>copia</i> para migrar (ver §1). Sigue funcionando con normalidad hasta el corte a Producción del §4.
Tu PC	Preparación y validación	Acá se hace TODO el trabajo de datos (export, carga, validación) con la copia de PC A, y se arma/prueba el .exe . Nada de esto se improvisa en la sesión remota.
PC B (remota, AnyDesk)	Destino del despliegue	Sólo PostgreSQL + el .exe empaquetado. Es un puesto único (confirmado) — no hay arquitectura de red multiusuario que resolver.

Esto simplifica bastante la sesión remota respecto a lo que se pensaba al principio: en PC B no hay usuarios que frenar ni [.mdb](#) que exportar en vivo — eso ya se resolvió en tu PC con una copia. La sesión por AnyDesk queda reducida a instalar y restaurar. Los certificados AFIP de Producción y Homologación ya están en tu poder (confirmado) — no hay ningún trámite pendiente que pueda demorar el corte del §4.

Índice

- [0. Respuesta directa: ¿conviene generar un .exe?](#)
- [1. Preparación previa — TODO esto se hace en tu PC, antes de conectar por AnyDesk](#)
- [2. Empaquetado: cómo generar el .exe](#)
- [3. El día de la migración remota — secuencia paso a paso](#)
- [4. Corte a Producción \(puede ser un día distinto\)](#)
- [5. Plan de rollback](#)
- [6. Preguntas que necesito que confirmes antes de fijar fecha](#)

0. Respuesta directa: ¿conviene generar un .exe?

Sí. Hoy el sistema se lanza con `Abrir FCMENU.vbs → .venv\Scripts\pythonw.exe -m migration.ui.main_menu` (ver el `.vbs` en la raíz del repo) — es decir, el "instalador" actual es literalmente la carpeta del proyecto con su entorno virtual de 336 MB expuesta tal cual, código fuente incluido y editable por cualquiera que abra la carpeta. Para una PC de operador esto es exactamente el riesgo que planteás: nada impide que alguien borre `migration/services.py` por error, mueva la carpeta, o rompa el `.venv` actualizando un paquete.

Recomendación concreta: empaquetar con PyInstaller en modo `--onedir` (no `--onefile`) y separar dos roles en la PC remota:

Carpeta	Contenido	Quién la usa	Permisos
<code>C:\FCMenu\</code>	El <code>.exe</code> empaquetado (onedir) + <code>.env</code> + certificados AFIP	El operador, todos los días	Sólo lectura/ejecución para el usuario operador (sin permiso de escritura/borrado — ver §2)
<code>C:\FCMenu-Admin\</code>	El repo fuente + venv de Python + Alembic (para vos)	Sólo vos, en sesiones de mantenimiento futuras	No expuesta al operador — ideal, ni siquiera queda instalada permanentemente (ver §3, Bloque J)

Por qué `--onedir` y no `--onefile`: un `.exe` de un solo archivo se auto-extrae a una carpeta temporal en **cada arranque** (más lento, notorio en una app PyQt6 con reportlab/cryptography/zeep adentro) y complica el antivirus corporativo de algunos clientes (falso positivo más frecuente en onefile que en onedir). `--onedir` arranca más rápido y es igual de "a prueba de operador" si además le sacás permisos de escritura a la carpeta.

Lo que el `.exe` NO reemplaza: seguridad de datos. El `.exe` evita que el operador borre el *código*, no reemplaza backups de la base ni permisos de carpeta bien puestos. Hacé las dos cosas — `exe` + `icacls` restringiendo la carpeta (ver §2) — no una en lugar de la otra.

Riesgo a verificar antes de comprometerte con esta vía: el `.venv` actual corre **Python 3.14.6**, una versión muy reciente. PyInstaller suele ir un paso atrás de las versiones de Python más nuevas. **Verificá localmente que `pyinstaller` empaqueta y corre sin errores contra 3.14.6 antes de prometerle una fecha al cliente** — si da problemas, la alternativa de 10 minutos es crear un venv de build aparte con Python 3.12 o 3.13 (ambas con soporte maduro en PyInstaller) sólo para el empaquetado, sin tocar el venv de desarrollo.

1. Preparación previa — TODO esto se hace en tu PC, antes de conectar por AnyDesk

No hagas nada de esto por primera vez con el cliente mirando la pantalla. Como PC B (la remota) es distinta de PC A (la del cliente, con el `fcmenu.mdb` real), **todo el trabajo de datos se resuelve acá, con una copia** — la sesión remota no incluye exportar Access ni coordinar un freeze de usuarios en PC B, eso ya no aplica con esta arquitectura.

1.1 Dos pasadas: Ensayo General y Pase Final

No lo hagas todo una sola vez:

- **Ensayo General** — con la copia de `fcmenu.mdb` que puedas conseguir ahora, corré el proceso completo de punta a punta para *probar que funciona* y dejar todo armado (paquetes, `.exe`, checklist). Sirve para encontrar y resolver problemas sin presión de tiempo ni cliente esperando.
- **Pase Final** — unos días antes de la fecha de despliegue acordada (cuanto más cerca, mejor), pedile al cliente una copia **fresca** de `fcmenu.mdb` desde PC A y repetí el mismo proceso ya probado. Al estar todo el camino resuelto en el Ensayo, esta segunda pasada debería tomar minutos, no horas. El resultado de este Pase Final (no el del Ensayo) es lo que se lleva a PC B.

Esto evita el problema de "drift": si preparás todo hoy pero el despliegue remoto es en tres semanas, en PC A se van a seguir cargando facturas/recibos/cobranzas todos los días — el Pase Final con una copia fresca es lo que mantiene los datos al día hasta el corte real.

1.2 Pasos (aplican tanto al Ensayo como al Pase Final)

1. **Commitear y etiquetar el estado actual del código** (una sola vez, no en cada pasada):

```
git add -A
git commit -m "Cierre de ETL repetible + corte de 5 años, previo a migración remota"
git tag v1.0-pre-migracion
git push origin main --tags
```

2. **Conseguir la copia de `fcmenu.mdb`** desde PC A (pendrive, Drive, lo que sea más práctico con el cliente) y guardarla con fecha en el nombre, ej. `fcmenu_2026-08-19.mdb` — nunca sobrescribas la copia anterior, dejá cada pasada trazable.
3. **Exportar Access → CSV**, localmente:

```
C:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe -File
migration\etl\export_access_to_csv.ps1
```

4. **Levantar un Postgres local temporal** para esta pasada (podés reusar `fcmenu_dev`, o crear `fcmenu_migracion` aparte para no mezclar con tus datos de desarrollo habituales) y aplicar el esquema:

```
alembic upgrade head
```

5. **Cargar los CSV:**

```
.venv\Scripts\python.exe migration\etl\load_csv_to_postgres.py
```

6. **Validar exhaustivamente antes de seguir** — con datos reales, no de prueba:
 - Conteo de filas por tabla contra lo esperado (`MovStock/Fcestad1` vacías por decisión ya tomada; el resto con el corte de 5 años aplicado).

- Elegir 3-5 clientes conocidos y comparar el saldo de cuenta corriente calculado contra lo que muestra hoy el legacy en PC A para esos mismos clientes — es la prueba real de que el "saldo anterior" sintético quedó bien calculado.

7. Generar el dump ya validado, listo para llevar a PC B:

```
pg_dump -Fc -h localhost -U fcmenu_app -d fcmenu_migracion -f  
fcmenu_prod.dump
```

Formato **-Fc** (custom): más chico y más rápido de restaurar que un **.sql** plano, y **pg_restore** recrea esquema + **alembic_version** + datos en un solo paso — en PC B ya no hace falta correr **alembic upgrade head** aparte.

8. **Correr la suite de tests completa** (**pytest**) — cero rojo antes de dar la pasada por buena.
9. **Armar y probar el .exe localmente** (ver §2) — abrir todas las ventanas principales, emitir una factura de prueba en **STUB** y, si es posible, una en **HOMOLOGACION** real desde el propio **.exe** (no sólo desde el venv de desarrollo) para confirmar que **cryptography/zeep** quedaron bien empaquetados. Esto es independiente de los pasos de datos — hazelo en paralelo la primera vez (Ensayo General), no hace falta repetirlo en el Pase Final salvo que haya cambiado el código.
10. **Armar los paquetes de transferencia** del Pase Final (más livianos que exportar en vivo en PC B):
 - **FCMenu-Operador.zip** — la carpeta **dist/** que genera PyInstaller (onedir).
 - **fcmenu_prod.dump** — el dump ya validado del Pase Final.
 - Instalador de **PostgreSQL 18 para Windows x64** (mismo mayor que ya validado en desarrollo).
 - Certificados AFIP de Producción y Homologación (ya los tenés) en un **.zip protegido con contraseña** (la contraseña se comunica por un canal separado — llamada o mensaje aparte, nunca junto con el archivo).
 - **FCMenu-Admin.zip** (repo fuente vía **git archive --format=zip -o FCMenu-Admin.zip HEAD**) — **opcional** para el despliegue inicial en sí (el dump ya trae esquema+datos, no hace falta el entorno Python del ETL en PC B para arrancar), pero conviene dejarlo preparado igual: cualquier actualización de esquema o recarga de datos futura lo va a necesitar.

2. Empaquetado: cómo generar el .exe

```
# En un venv de build (nuevo o el mismo si 3.14 anda bien con PyInstaller)
.venv/Scripts/python.exe -m pip install pyinstaller

.venv/Scripts/pyinstaller `
  --name FCMenu `
  --windowed `
  --onedir `
  --icon ICON1.ICO `
  --add-data "assets;assets" `
  --hidden-import zeep `
  --hidden-import lxml `
  --hidden-import cryptography `
  --collect-all reportlab `
  --collect-all PyQt6 `
  migration/ui/main_menu.py
```

Notas concretas sobre este comando:

- `--windowed`: evita que se abra una consola negra detrás de la app (mismo motivo por el que hoy existe `Abrir FCMENU.vbs` — replicó esa experiencia).
- `--collect-all PyQt6`: PyInstaller a veces no detecta solo los plugins de plataforma de Qt (`platforms/qwindows.dll`) — sin esto, el `.exe` puede compilar pero fallar al abrir con un error de "no se pudo cargar la plataforma windows". Probalo igual localmente: si falla, agregar explícitamente `--add-binary` apuntando a la carpeta `platforms` del PyQt6 instalado.
- `--hidden-import zeep/lxml/cryptography`: estas tres son las que más problemas dan con PyInstaller por importación dinámica interna — mejor declararlas explícitas que descubrirlo con el cliente mirando.
- `--icon ICON1.ICO`: ya existe en la raíz del repo (el ícono del sistema legacy) — reusalo para continuidad visual con el acceso directo de siempre.
- **Probá el resultado en una PC limpia** (una VM sin Python instalado, si tenés) antes de llevarlo a la PC del cliente — es la única forma real de confirmar que no falta ninguna dependencia del sistema.

Después de instalar en la PC del cliente, restringí permisos de la carpeta del operador (evita ediciones/borrados accidentales, complementa al exe):

```
icacls "C:\FCMenu" /inheritance:r
icacls "C:\FCMenu" /grant:r "Users:(OI)(CI)RX"
icacls "C:\FCMenu" /grant:r "Administrators:(OI)(CI)F"
```

Esto deja a cualquier usuario estándar con permiso de **lectura y ejecución únicamente** sobre `C:\FCMenu` — puede correr la app, no puede borrar ni modificar archivos. Los administradores (vos, vía AnyDesk con la cuenta correspondiente) mantienen control total para actualizaciones futuras.

3. El día de la migración remota — secuencia paso a paso

Con el trabajo de datos ya resuelto en §1 (Pase Final), esta sesión en PC B es corta: instalar Postgres, restaurar el dump, instalar el `.exe`, validar. **No hay freeze de usuarios ni exportación de Access que hacer acá** — PC

B nunca tuvo el sistema legacy, y PC A (donde sí está) no se toca hasta el corte a Producción del §4.

Bloque A — Reconocimiento (10 min)

1. Conectar por AnyDesk con una cuenta con permisos de administrador de Windows en PC B (necesario para instalar PostgreSQL y el servicio).
2. Verificar espacio en disco libre — con esta arquitectura alcanza con menos margen que si se exportara en vivo ahí: PostgreSQL (~500 MB instalado) + el dump + el **.exe** empaquetado. **2 GB libres** son de sobra.
3. Verificar que el puerto 5432 esté libre (no hay otro PostgreSQL/servicio ya escuchando ahí en PC B).

Bloque B — Instalar PostgreSQL (10-15 min)

1. Correr el instalador de PostgreSQL 18 (ya bajado con anticipación, ver §1.2).
2. Definir contraseña del superusuario **postgres** — guardarla en un gestor de contraseñas, no en un **.txt** en el escritorio.
3. Puerto default **5432**, sin necesidad de Stack Builder.
4. Confirmar que el servicio de Windows (**postgresql-x64-18**) quedó en arranque automático.
5. **No** exponer el puerto 5432 a la red (dejar **pg_hba.conf** en su default de sólo-localhost) — puesto único (confirmado), Postgres y app en la misma PC B, no hace falta acceso remoto a la base y cada puerto abierto de más es superficie de ataque innecesaria en una PC a la que también entrás por AnyDesk.

Bloque C — Restaurar el dump (5-10 min)

1. Crear el rol de aplicación (privilegio mínimo, no superusuario) y la base vacía:

```
CREATE ROLE fcmenu_app LOGIN PASSWORD 'CONTRASEÑA_FUERTE_NUEVA';  
CREATE DATABASE fcmenu_prod OWNER fcmenu_app;
```

2. Transferir **fcmenu_prod.dump** a PC B (transferencia de archivos nativa de AnyDesk — el dump comprimido debería ser bastante más chico que los CSV sueltos).
3. Restaurar todo de una vez (esquema + **alembic_version** + datos, ya validado en §1.2):

```
pg_restore -h localhost -U fcmenu_app -d fcmenu_prod --no-owner  
fcmenu_prod.dump
```

4. Verificación rápida de conteo de filas por tabla — no hace falta repetir la validación exhaustiva del §1.2 (ya se hizo sobre estos mismos datos), esto es sólo confirmar que la transferencia/restauración no corrompió nada.

Bloque D — Configurar la app del operador (10 min)

1. Transferir **FCMenu-Operador.zip** (el **.exe** empaquetado) a **C:\FCMenu** y descomprimir.
2. Transferir los certificados AFIP (el **.zip** con contraseña preparado en §1.2) a una carpeta fija, por ejemplo **C:\FCMenu\certs** — **borrar el .zip del disco apenas se extrae**, no dejarlo dando vueltas.

3. Crear el `.env` de producción en `C:\FCMenu\`:

```
DATABASE_URL=postgresql+psycopg2://fcmenu_app:CONTRASEÑA_FUERTE_NUEVA@localhost:5432/fcmenu_prod
FCMENU_AFIP_ENTORNO=HOMOLOGACION
FCMENU_AFIP_CUIT=33703467909
FCMENU_AFIP_CERT_PATH=C:\FCMenu\certs\homologacion.crt
FCMENU_AFIP_KEY_PATH=C:\FCMenu\certs\homologacion.key
```

Arrancar siempre primero en HOMOLOGACION, nunca directo en PRODUCCION — es el paso de validación del Bloque E.

4. Aplicar los permisos restrictivos de carpeta del §2 (`icacls`).
5. Crear el acceso directo de escritorio apuntando al nuevo `.exe` — como PC B nunca tuvo el legacy instalado, no hay acceso directo viejo con el que convivir acá (a diferencia de PC A, ver §4).

Bloque E — Validación funcional (30-60 min — no apurar este paso)

Checklist mínimo antes de considerar el sistema "listo":

- ☐ `MainMenuWindow` abre y el cartel de entorno muestra correctamente "Homologación / Prueba".
- ☐ Búsqueda de Cliente y de Artículo devuelven datos reales y consistentes con lo que el operador reconoce.
- ☐ Emitir una Factura de prueba completa de punta a punta en Homologación — confirma el circuito real: TRA → firma CMS → SOAP AFIP → CAE aprobado → PDF con QR → impresión.
- ☐ El stock del artículo facturado se descontó correctamente.
- ☐ El saldo de cuenta corriente del cliente se actualizó correctamente.
- ☐ Emitir un Recibo de prueba, con al menos un pago imputado a la factura recién emitida.
- ☐ Abrir Listados/Consultas y confirmar que los reportes traen datos.
- ☐ No hace falta correr `pytest` en PC B — la suite usa SQLite en memoria por diseño (ver Cap. 4.4 de la documentación de arquitectura) y ya corrió en tu PC en el §1.2, paso 8. Si en algún momento llevás `FCMenu-Admin` a PC B para una tarea de mantenimiento futura, ahí sí podés correrla como chequeo de que ese entorno Python quedó sano — nunca como prueba contra `fcmenu_prod`.

4. Corte a Producción (puede ser un día distinto)

No hace falta que esto pase el mismo día que la instalación — de hecho, **es preferible dejar unos días de convivencia** con el sistema en Homologación mientras el cliente lo prueba con confianza, antes de emitir comprobantes fiscales reales.

1. Antes de este paso, hacé un **Pase Final actualizado** (§1.1) si pasó tiempo desde el que se usó para poblar PC B — no emitas el primer comprobante real de Producción sobre datos que puedan haber quedado desactualizados respecto a PC A.
2. Reemplazar en `.env` de PC B: `FCMENU_AFIP_ENTORNO=PRODUCCION`, y las rutas de certificado/clave por el par de **Producción** (distinto del de Homologación — ya lo tenés).
3. Reiniciar la app y confirmar que el cartel ahora muestra **"PRODUCCIÓN"** — verificación visual obligatoria antes de emitir nada.

4. Emitir el primer comprobante real con supervisión directa (vos monitoreando por AnyDesk mientras el operador lo hace, o haciéndolo vos mismo la primera vez) — confirmar CAE real aprobado y que el comprobante aparece en el portal de AFIP del cliente.
5. Recién ahí, comunicar formalmente al equipo del cliente que a partir de ahora se factura desde PC B — y coordinar qué pasa con PC A (ver §5).

5. Plan de rollback

Esta arquitectura de tres máquinas te da una ventaja de rollback que no tenías en la versión anterior de este plan: **PC A (la real) nunca se tocó** en todo el proceso — sólo se le sacaron copias de `fcmenu.mdb`, nunca se escribió nada ahí ni se instaló nada nuevo. Si algo sale mal en cualquier punto de §1 a §3, no hay nada que revertir en PC A: el cliente sigue facturando ahí con normalidad mientras se resuelve el problema en PC B con calma.

Si algo sale mal **después** del corte a Producción del §4 (peor caso, y el único que realmente importa como riesgo), la reversión ya no es trivial porque puede haber CAE reales emitidos desde PC B que no existen en el `.mdb` de PC A — por eso el Bloque E de validación en Homologación no se acorta nunca, y por eso conviene dejar pasar unos días de uso real en Homologación en PC B antes de tocar Producción. Mitigación adicional: no dar de baja PC A el mismo día del corte — dejarla intacta y disponible (aunque ya no se cargue nada ahí) al menos 2-4 semanas después del corte a Producción, por si hace falta consultar algo del historial viejo mientras el equipo se acostumbra al nuevo sistema.

6. Estado de las definiciones

Ya confirmado (no requiere más consulta):

Punto	Definición
PC de destino	Distinta de la que corre el legacy — PC A queda intacta (§ arquitectura, portada)
Certificados AFIP (Homologación y Producción)	Ya en tu poder — sin trámite pendiente que bloquee el §4
Cantidad de puestos	Uno solo — Postgres local en PC B, sin arquitectura de red multiusuario
Conectividad de PC B para AnyDesk	Muy buena — la transferencia de archivos en vivo (dump, <code>.exe</code> , certificados) no debería ser un cuello de botella; igual conviene dejar pre-cargado el instalador de PostgreSQL con anticipación por ser el archivo más pesado (~300 MB)

Lo único que falta acordar para poner fecha:

- **¿Cuándo hacemos el Ensayo General?** — se puede arrancar ya, con cualquier copia de `fcmenu.mdb` que consigas, sin depender de coordinar nada con el cliente todavía.
- **¿Cómo vas a conseguir la copia fresca de `fcmenu.mdb` para el Pase Final** (unos días antes de la fecha real) — **el cliente te la manda, o tenés alguna forma de generarla vos mismo remotamente** (ej. otra sesión corta de AnyDesk a PC A sólo para copiar el archivo, sin instalar nada ahí)?